SPEAKMATE

Feature Specification

STORY BANK

Ngân hàng câu chuyện cá nhân

Phiên bản:	1.0
Ngày:	19/04/2026
Đội ngũ:	NEXT GENTS
Trạng thái:	Draft - Chờ review

1. Tổng quan tính năng

Mục tiêu cốt lõi

Story Bank giải quyết rào cản "nghẽn cổ chai thông tin" - người dùng có trải nghiệm, ý tưởng trong đầu nhưng không biết cách diễn đạt thành lời nói mạch lạc. Story Bank giúp họ đóng gói sẵn những ý tưởng đó thành các mẩu chuyện ngắn gọn, có cấu trúc, để khi cần giao tiếp thì truy xuất ra nói ngay.

1.1. Vị trí trong hệ thống SpeakMate

Story Bank thuộc về phần 30% chuẩn bị trong nguyên lý 30/70 của SpeakMate. Nó là phòng riêng biệt, tách hoàn toàn với Phòng gym luyện tập (70%).

Phòng Story Bank (30%)	Phòng gym (70%)	Bridge to Reality
Chuẩn bị chất liệu giao tiếp	Luyện tập nói trong kịch bản	Áp dụng ngoài đời thực
Input: trải nghiệm thô của user	Input: Story Bank + kịch bản AI	Input: nhiệm vụ từ Mentor Ni
Output: mẩu chuyện có cấu trúc	Output: báo cáo đánh giá	Output: phản hồi thực chiến

1.2. Flow kết nối với các phòng khác

Phòng Story Bank (xây kho chất liệu) → Trước phiên gym (hệ thống gợi ý story phù hợp, user ôn lại) → Phòng gym (nói tự nhiên, không hỗ trợ gì) → Báo cáo sau phiên (chỉ ra story nào chưa dùng, phần nào bỏ sót) → Cập nhật Story Bank → Bridge to Reality (nhiệm vụ ngoài đời) → Quay lại cập nhật story mới nếu có trải nghiệm mới.

2. User workflow chi tiết

2.1. Luồng tạo Story mới

Bước 1: Nhập chất liệu thô

User vào phòng Story Bank, chọn "Tạo mới". Có 3 cách nhập:

- **Voice input:** Nói trực tiếp với Mentor Ni, hệ thống dùng PhoWhisper chuyển thành text. Đây là cách tự nhiên nhất.
- **Text input:** Gõ bullet points hoặc đoạn văn lộn xộn vào khung text.
- **Upload tài liệu:** Đưa CV, file ghi chú, mô tả dự án. AI trích xuất và gợi ý story tiềm năng.

> **Ví dụ input thô từ user**
> "em biết làm backend" / "đồ án môn CNPM dùng Node.js" / "tự deploy lên VPS" / "bị lỗi database lúc demo, sửa được trong 10 phút"

Bước 2: Mentor Ni hỏi bổ sung

Mentor Ni phân tích input, xác định thông tin còn thiếu, hỏi tối đa 2 câu bổ sung. Ví dụ: "Đồ án đó có bao nhiêu người?", "Kết quả cuối cùng thế nào?". Mục tiêu: thu thập đủ thông tin cho framework STAR mà không làm user mệt.

Bước 3: AI cấu trúc hóa

Gọi Gemini 2.0 Flash với prompt yêu cầu cấu trúc hóa theo framework phù hợp (STAR mặc định, hoặc PREP cho tranh biện, CAR cho resume). Output là JSON theo schema định sẵn, qua Validation Gate trước khi hiện cho user.

> **Ví dụ output sau cấu trúc hóa**
> Situation: Đồ án nhóm 4 người, môn CNPM, dùng Node.js. | Task: Phụ trách toàn bộ API và database, tự deploy lên VPS. | Action: Database bị lỗi kết nối ngay lúc demo trước giảng viên, debug và sửa tại chỗ. | Result: Sửa trong 10 phút, nhóm được 9 điểm. | Thời lượng nói ước tính: ~40 giây.

Bước 4: User review và lưu

User xem preview story đã cấu trúc, chỉnh sửa cho đúng giọng mình (AI viết formal quá thì sửa lại cho tự nhiên). Gắn tags tự động + tùy chỉnh. Lưu vào Story Bank.

2.2. Luồng quản lý Story Bank

- Xem tất cả story, lọc theo tag, tìm kiếm theo từ khóa

- Mỗi story hiện trạng thái: chưa luyện / đã luyện X lần / điểm đánh giá gần nhất
- Story luyện nhiều + điểm cao: đánh dấu "sẵn sàng thực chiến"
- Mentor Ni gợi ý định kỳ: "Bạn có 5 story về kỹ thuật nhưng chưa có story về teamwork"
- Chỉnh sửa, cập nhật, xóa story bất kỳ lúc nào

2.3. Luồng kết nối với Phòng gym

Trước phiên luyện tập

User chọn bối cảnh trong Phòng gym (ví dụ: "Phỏng vấn Backend Developer"). Hệ thống phân tích bối cảnh, query Story Bank theo tags, hiện danh sách: "Bạn có 3 story phù hợp cho buổi này". User ôn nhanh rồi bấm Bắt đầu.

Trong phiên luyện tập

Không có gợi ý Story Bank nào xuất hiện. User phải tự nhớ và tự nói. Đúng triết lý 70% thực hành thuần túy.

Sau phiên luyện tập

Analyst so sánh transcript thực tế với Story Bank gốc. Báo cáo chỉ ra: phần nào đã nói được, phần nào bỏ sót (ví dụ: quên Result), story nào phù hợp mà chưa dùng. Gợi ý cập nhật Story Bank.

3. Cấu trúc dữ liệu

3.1. StoryEntry schema

Field	Type	Mô tả
`id`	string	UUID, primary key
`userId`	string	FK tới bảng users
`title`	string	Tiêu đề ngắn, ví dụ: "Fix lỗi DB lúc demo"
`rawInput`	string	Bullet points / transcript gốc user nhập
`inputMethod`	enum	'voice' | 'text' | 'upload'
`framework`	enum	'STAR' | 'PREP' | 'CAR'
`structured`	JSON	Object chứa situation, task, action, result (hoặc tương đương theo framework)
`fullScript`	string	Đoạn nói hoàn chỉnh 30-60 giây, ngôn ngữ tự nhiên
`estimatedDuration`	number	Thời lượng nói ước tính (giây)
`tags`	string[]	Tags phân loại, ví dụ: ["backend", "áp lực", "teamwork"]
`status`	enum	'draft' | 'ready' | 'battle-tested'
`practiceCount`	number	Số lần đã dùng trong Phòng gym
`lastScore`	number | null	Điểm coverage gần nhất (0-100)
`createdAt`	timestamp	Thời điểm tạo
`updatedAt`	timestamp	Thời điểm cập nhật gần nhất

3.2. PracticeHistory schema (bảng con)

Field	Type	Mô tả
`id`	string	UUID
`storyId`	string	FK tới StoryEntry
`sessionId`	string	FK tới phiên luyện tập trong Phòng gym
`coverageScore`	number	% nội dung Story Bank được nói trong phiên (0-100)
`missedParts`	string[]	Phần nào bị bỏ sót, ví dụ: ["result"]
`addedParts`	string[]	Phần nào user nói thêm ngoài Story Bank
`feedback`	string	Nhận xét từ Analyst
`date`	timestamp	Thời điểm phiên luyện tập

4. Luồng kỹ thuật

4.1. Tạo Story mới

1. User nhập text hoặc nói → nếu voice thì PhoWhisper (local) chuyển thành text
2. Gửi raw text lên backend (Control Plane)
3. Backend gọi Gemini 2.0 Flash API:

```
Prompt: "Người dùng muốn chuẩn bị một mẩu chuyện để
giao tiếp. Dưới đây là ý tưởng thô. Hãy:
1) Hỏi tối đa 2 câu bổ sung nếu thiếu thông tin
2) Cấu trúc hóa theo STAR
3) Viết đoạn nói tự nhiên 30-60 giây
4) Gợi ý tags phù hợp
Trả về JSON theo schema: {StoryEntry}"
```

4. Validation Gate (TypeScript/Zod) kiểm tra output đúng schema
5. Nếu Mentor Ni cần hỏi bổ sung: trả về câu hỏi cho user, user trả lời, gọi Gemini lần 2 với đầy đủ context
6. User preview, chỉnh sửa → confirm → lưu Supabase

4.2. Gợi ý Story trước phiên gym

Cơ chế: Tag matching (không dùng AI, instant)

1. User chọn bối cảnh trong Phòng gym
2. The Brain (Gemini 2.0 Flash) tạo kịch bản, mỗi câu hỏi có field relevantTags
3. Frontend query Story Bank local: filter stories theo tags trùng khớp
4. Hiện danh sách gợi ý: chỉ tiêu đề + tag + trạng thái luyện tập
5. User ôn nhanh → bấm Bắt đầu → vào phòng gym (không còn gợi ý)

4.3. Phân tích sau phiên gym

1. Analyst (Gemini 2.0 Pro) nhận: transcript phiên + Story Bank entries có tag trùng
2. Prompt phân tích: so sánh câu trả lời thực tế vs. bản Story Bank đã chuẩn bị
3. Output: coverageScore, missedParts, addedParts, feedback, gợi ý cập nhật Story Bank
4. Cập nhật PracticeHistory → cập nhật practiceCount và lastScore trong StoryEntry
5. Nếu user nói tốt hơn bản Story Bank: gợi ý "Câu trả lời thực tế hay hơn bản chuẩn bị, cập nhật Story Bank?"

5. Thiết kế prompt cho Mentor Ni

5.1. Prompt tạo Story Bank

System prompt cho Mentor Ni trong phòng Story Bank

Bạn là Ni, mentor giao tiếp cá nhân. Nhiệm vụ: giúp user đóng gói trải nghiệm thành mẩu chuyện có cấu trúc. Quy tắc: (1) Lắng nghe input thô, không phán xét. (2) Nếu thiếu thông tin quan trọng, hỏi TỐI ĐA 2 câu ngắn gọn. (3) Cấu trúc hóa theo framework phù hợp nhất. (4) Viết đoạn nói 30-60 giây bằng giọng tự nhiên, đúng lứa tuổi 15-22. (5) Gợi ý tags và tình huống có thể dùng. (6) Trả về JSON theo schema đã định.

5.2. Prompt phân tích sau phiên gym

System prompt cho Analyst khi so sánh Story Bank

Bạn nhận được: (A) Story Bank entry gốc đã chuẩn bị, (B) Transcript thực tế từ phiên luyện tập. Nhiệm vụ: (1) So sánh từng phần STAR: user đã cover chưa, mức độ chi tiết so với bản gốc. (2) Tính coverageScore (0-100). (3) Liệt kê missedParts. (4) Nếu user nói thêm nội dung hay ngoài Story Bank, ghi nhận vào addedParts. (5) Gợi ý cập nhật Story Bank cụ thể. Quy tắc: PHẢI trích dẫn chính xác từ transcript làm bằng chứng.

6. Hướng dẫn thiết kế UI

6.1. Phòng Story Bank - Màn hình chính

- Header: "Ngân hàng câu chuyện" + nút "Tạo mới"
- Thanh filter: tags (dạng chip), trạng thái (tất cả / chưa luyện / sẵn sàng thực chiến)
- Danh sách story dạng card, mỗi card hiển thị: tiêu đề, tags (chip màu), framework (STAR/PREP/CAR), trạng thái luyện tập (chưa luyện / đã luyện X lần / điểm gần nhất), thời lượng nói ước tính
- Empty state (lần đầu): Mentor Ni chào và hướng dẫn tạo story đầu tiên

6.2. Màn hình tạo Story mới

- Phía trái: khung nhập liệu (text area + nút voice input + nút upload)
- Phía phải: preview story đã cấu trúc (realtime cập nhật khi AI xử lý)
- Phía dưới: tags gợi ý (chip, có thể thêm/xóa) + nút Lưu
- Nếu Mentor Ni cần hỏi bổ sung: hiện chat bubble nhỏ giữa màn hình

6.3. Màn hình chi tiết Story

- Hiển thị đầy đủ story theo cấu trúc STAR (hoặc framework tương ứng)
- Đoạn fullScript với nút "Nghe thử" (TTS đọc lại cho user nghe)
- Lịch sử luyện tập: timeline các phiên đã dùng story này, điểm coverage theo thời gian
- Nút "Chỉnh sửa" và "Luyện tập ngay" (nhảy thẳng vào Phòng gym với bối cảnh phù hợp)

6.4. Gợi ý trước phiên gym

- Popup/modal xuất hiện sau khi user chọn bối cảnh, trước khi bấm Bắt đầu
- Tiêu đề: "Story Bank gợi ý cho buổi này"
- Danh sách 1-3 story phù hợp: hiện tiêu đề + tag + nút "Ôn nhanh" (mở chi tiết story)
- Nút "Bỏ qua, bắt đầu luôn" cho user không muốn ôn

7. Acceptance criteria

#	Tiêu chí	Ưu tiên	Trạng thái
1	User tạo được story mới bằng text input	P0 - Bắt buộc	Chưa làm
2	User tạo được story mới bằng voice input	P0 - Bắt buộc	Chưa làm
3	AI cấu trúc hóa input thành STAR format + fullScript	P0 - Bắt buộc	Chưa làm
4	Mentor Ni hỏi bổ sung (tối đa 2 câu) khi thiếu thông tin	P0 - Bắt buộc	Chưa làm
5	User chỉnh sửa, gắn tag, lưu story thành công	P0 - Bắt buộc	Chưa làm
6	Danh sách Story Bank với filter theo tag và trạng thái	P0 - Bắt buộc	Chưa làm
7	Gợi ý Story Bank trước phiên gym (tag matching)	P1 - Quan trọng	Chưa làm
8	Analyst so sánh transcript vs Story Bank sau phiên gym	P1 - Quan trọng	Chưa làm
9	Coverage score + missed parts trong báo cáo	P1 - Quan trọng	Chưa làm
10	Upload CV/tài liệu để trích xuất story tự động	P2 - Nice to have	Chưa làm
11	Nút "Nghe thử" TTS cho fullScript	P2 - Nice to have	Chưa làm
12	Hỗ trợ framework PREP và CAR ngoài STAR	P2 - Nice to have	Chưa làm

8. Lưu ý quan trọng cho dev

> **Nguyên tắc thiết kế cốt lõi**
> Story Bank là KHO CHẤT LIỆU CÁ NHÂN do user tự xây, không phải kịch bản AI tạo sẵn. AI chỉ giúp cấu trúc hóa, không bịa thêm nội dung. Mọi story phải đến từ trải nghiệm thật của user.

8.1. Ranh giới Story Bank vs. Phòng gym

- Story Bank: chuẩn bị TRƯỚC khi luyện. User chủ động, có thời gian suy nghĩ.
- Phòng gym: luyện tập TRONG kịch bản. User bị áp lực thời gian, phải phản xạ.
- KHÔNG hiện nội dung story trong phòng gym. Chỉ gợi ý tiêu đề TRƯỚC phiên.

8.2. Privacy

- Story Bank chứa thông tin cá nhân nhạy cảm (trải nghiệm, dự án, điểm số).
- Áp dụng nguyên tắc Privacy First: dữ liệu Story Bank chỉ lưu trên Supabase, mã hóa at rest.
- Khi gọi Gemini API để cấu trúc hóa: không gửi userId hay thông tin định danh.
- User có quyền xóa toàn bộ Story Bank bất kỳ lúc nào.

8.3. Performance

- Tạo story mới: target < 5 giây từ lúc user confirm đến khi hiện preview.
- Tag matching cho gợi ý: phải instant (< 100ms), chạy hoàn toàn client-side.
- Danh sách Story Bank: load < 1 giây cho 50 stories.

8.4. Validation

- Mọi output từ Gemini phải qua Validation Gate (Zod schema).
- Nếu AI trả về sai schema: retry tối đa 2 lần, sau đó báo lỗi cho user.
- fullScript không được vượt quá 200 từ (tương đương ~60-90 giây nói).
- Tags tối đa 10 tags/story, mỗi tag tối đa 30 ký tự.